

Department of Computer Engineering
CMSE318 – Principles of Programming Languages



Assignment 2

Group #2

Team #9

Hani Habib 23701447

Hasan Altayeb 23703026

SPRING SEMESTER 2025-2026

A) Aim of the program

This program is a lexical analyzer that reads a text file with code written in Python and breaks it down into separate tokens. It uses a symbol value and a set of rules that have already been set up to find different types of tokens, like integers, floating point numbers, valid identifiers, and keywords. The lexer gives back a LexResult object for each token it looks at. This object has information about the token's type and any values that go with it.

Program Code:

```
relops=['<=', '<', '>', '>=', '==', '!=']
i=0
data=[]
lexemesTable={
    "Lexemes": "Tokens",
    "FOR": "FOR_TOKEN",
    "WHILE": "WHILE_TOKEN",
    "IF": "IF_TOKEN"
}

def is_int(nums):
    try:
        int(nums)
        return True
    except ValueError:
        return False

def is_float(nums):
    try:
        float(nums)
        return True
    except ValueError:
        return False

def special_char(string):
    spe_char_set=set('!@#$%^&*()+{}:"<>?[];\\".,/'~-=')
    return bool(set(string) & spe_char_set)

def lex():
    global i,data,lexemesTable
    if i<len(data):
        if data[i]=="if":
            print(f"Token: IF_TOKEN\nLexeme: {data[i]}")
            lexemesTable.update({data[i]: "IF_TOKEN"})
        elif data[i]=="while":
            print(f"Token: WHILE_TOKEN\nLexeme: {data[i]}")
            lexemesTable.update({data[i]: "WHILE_TOKEN"})
        elif data[i]=="for":
            print(f"Token: FOR_TOKEN\nLexeme: {data[i]}")
            lexemesTable.update({data[i]: "FOR_TOKEN"})
        elif data[i] in relops:
            print(f"Token: RELOP\nLexeme: {data[i]}")
            lexemesTable.update({data[i]: "RELOP"})
```

```

elif 'a'<=data[i][0].lower()<='z' and not special_char(data[i]):
    print(f"Token: ID\nLexeme: {data[i]}")
    lexemesTable.update({data[i]: "ID"})
elif is_int(data[i]):
    print(f"Token: INTEGER\nLexeme: {data[i]}")
    lexemesTable.update({data[i]: "INTEGER"})
elif is_float(data[i]):
    print(f"Token: FLOAT\nLexeme: {data[i]}")
    lexemesTable.update({data[i]: "FLOAT"})
else:
    print(f"Token: ERROR\nLexeme: {data[i]}")
    i+=1
else:
    print("No more lexemes.")
return

while True:
    f=input("Enter the filename:")
    try:
        with open(f, "r") as l:
            for line in l:
                for word in line.split():
                    data.append(word)
            print("File loaded sucessfully")
    except:
        print("Error reading file. Try again.\n")
        continue
    break

while True:
    choice=input("\n\nChoose an option\n1- Call Lex()\n2- Show Symbol Table\n3- Exit\n")
    if choice=='1':
        lex()
    elif choice=='2':
        for key, value in lexemesTable.items():
            print("{:<10} {:<10}".format(key,value))
    elif choice=='3':
        print("Goodbye!")
        exit()

```

~\lexemes.txt

while > m\$oney _something while <= if -2.9 < -39 -python 65the 87 >= for hello for woRld 94HGE 18.74 == if
th_ink

B) Functionalities of the code:

1) Global Variables

```
relops=['<=', '<', '>', '>=', '==', '!=']
i=0
data=[]
lexemesTable={
    "Lexemes": "Tokens",
    "FOR": "FOR_TOKEN",
    "WHILE": "WHILE_TOKEN",
    "IF": "IF_TOKEN"
}
```

A list of relational operators and a symbol table that stores important words (like identifiers and keywords).

2) Functions

Helper Functions:

```
10
11     ~ def is_int(nums):
12         ~ try:
13             ~ int(nums)
14             ~ return True
15         ~ except ValueError:
16             ~ return False
17
18     ~ def is_float(nums):
19         ~ try:
20             ~ float(nums)
21             ~ return True
22         ~ except ValueError:
23             ~ return False
24
25     ~ def special_char(string):
26         spe_char_set=set('@#$$%^&*()+{}:"<>?[];\'\\". / ~ - ')
27         ~ return bool(set(string) & spe_char_set)
28
29     ~ def lex():
30         ~ global i, data, lexemesTable
31         ~ if i<len(data):
32             ~ if data[i]=="if":
33                 ~ print(f"Token: IF_TOKEN\nLexeme: {data[i]}")
34                 ~ lexemesTable.update({data[i]: "IF_TOKEN"})
35             ~ elif data[i]=="while":
36                 ~ print(f"Token: WHILE_TOKEN\nLexeme: {data[i]}")
37                 ~ lexemesTable.update({data[i]: "WHILE_TOKEN"})
38             ~ elif data[i]=="for":
39                 ~ print(f"Token: FOR_TOKEN\nLexeme: {data[i]}")
40                 ~ lexemesTable.update({data[i]: "FOR_TOKEN"})
41             ~ elif data[i] in relops:
42                 ~ print(f"Token: RELOP\nLexeme: {data[i]}")
43                 ~ lexemesTable.update({data[i]: "RELOP"})
44             ~ elif 'a'<=data[i][0].lower()<='z' and not special_char(data[i]):
45                 ~ print(f"Token: ID\nLexeme: {data[i]}")
46                 ~ lexemesTable.update({data[i]: "ID"})
47             ~ elif is_int(data[i]):
48                 ~ print(f"Token: INTEGER\nLexeme: {data[i]}")
49                 ~ lexemesTable.update({data[i]: "INTEGER"})
50             ~ elif is_float(data[i]):
51                 ~ print(f"Token: FLOAT\nLexeme: {data[i]}")
52                 ~ lexemesTable.update({data[i]: "FLOAT"})
53             ~ else:
54                 ~ print(f"Token: ERROR\nLexeme: {data[i]}")
55             ~ i+=1
56         ~ else:
57             ~ print("No more lexemes.")
```

is_int(): checks if something is an integer

is_float(): checks if something is a float

special_char(): checks if a word contains invalid symbols

The lex() Function: Looks at the current word, Checks what it is (keyword, number, ID, operator, etc.)

Prints: the token type, the actual word (lexeme), and Moves to the next word.

File Reading:

```
61     while True:
62         f=input("Enter the filename:")
63         try:
64             with open(f, "r") as l:
65                 for line in l:
66                     for word in line.split():
67                         data.append(word)
68             print("File loaded sucessfully")
69         except:
70             print("Error reading file. Try again.\n")
71             continue
72         break
```

First, asks the user for a file name to input, then opens the file, reads all words, and stores them in data.

Menu:

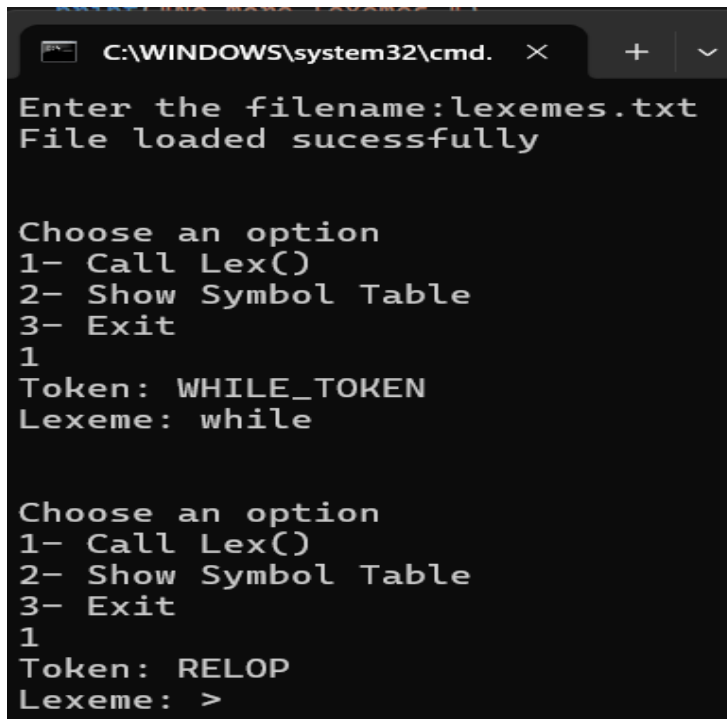
```
74     while True:
75         choice=input("\n\nChoose an option\n1- Call Lex()\n2- Show Symbol Table\n3- Exit\n")
76         if choice=='1':
77             lex()
78         elif choice=='2':
79             for key, value in lexemesTable.items():
80                 print("{:<10} {:<10}".format(key,value))
81         elif choice=='3':
82             print("Goodbye!")
83             exit()
```

The program keeps on running in a loop and gives 3 options:

- 1) Call lex() → which will process one word at a time.
- 2) Show Symbol Table → which will display stored identifiers and keywords.
- 3) Exit → Ends the program.

C) Sample Run of the program:

Screenshot(1): Entered a file name to read from, then chose the first option to check the first word, and ran it again to check the second word.



```
C:\WINDOWS\system32\cmd. x + v
Enter the filename:lexemes.txt
File loaded sucessfully

Choose an option
1- Call Lex()
2- Show Symbol Table
3- Exit
1
Token: WHILE_TOKEN
Lexeme: while

Choose an option
1- Call Lex()
2- Show Symbol Table
3- Exit
1
Token: RELOP
Lexeme: >
```

Screenshot(2): After checking all the words in the file, chose option 2 to display all the stored Lexemes and their tokens.

```
C:\WINDOWS\system32\cmd. X +
1- Call Lex()
2- Show Symbol Table
3- Exit
2
Lexemes      Tokens
FOR          FOR_TOKEN
WHILE        WHILE_TOKEN
IF           IF_TOKEN
while        WHILE_TOKEN
>            RELOP
<=          RELOP
if           IF_TOKEN
-2.9         FLOAT
<            RELOP
-39          INTEGER
87           INTEGER
>=          RELOP
for          FOR_TOKEN
hello        ID
world        ID
18.74        FLOAT
==          RELOP
th_ink       ID

Choose an option
1- Call Lex()
2- Show Symbol Table
3- Exit
```

Screenshot(3): chose option 3 to exit the program.

```
Choose an option
1- Call Lex()
2- Show Symbol Table
3- Exit
3
Goodbye!
Press any key to continue . . . |
```

Conclusion

This assignment demonstrated how a lexical analyzer works by reading input and classifying lexemes into tokens. It helped reinforce key programming concepts such as string processing, condition checking, and data structures like symbol tables. Overall, it provided a clear understanding of the first stage of language processing in compilers.